

Movie-Spoiler detector

Rahul Shah
From IIT Delhi
May-Batch

May 2025

1 Introduction

User-generated reviews are often our first point of contact when we consider watching a movie or a TV show. However, beyond telling us the qualitative aspects of the item we want to consume, reviews may inevitably contain undesired revelatory information (i.e. 'spoilers') such as the surprising fate of a character in a movie, or identity of a murderer in a crime-suspense movie etc. For users who are interested in consuming the item but are unaware of the critical plot twists, spoilers may decrease the excitement regarding the pleasurable uncertainty and curiosity of media consumption. Therefore, a natural question is how to identify these spoilers in entertainment reviews, so that users can more effectively navigate review platforms.

2 About data

About dataset:

Statistics	Values
Reviews	573,913
Spoiler reviews	150,924
Users	263,407
Movies	1,572
Users with at least one movie spoiler	79,039
Movies with at least one spoiler	1,570

Table 1: Dataset Statistics for Spoiler Detection

The dataset is divided into 2 files: IMDB_movie_details contains metadata about the movies in the dataset and IMDB_reviews contains user reviews to the movies available in the dataset.

Each record in IMDB_movie_details consists of the following attributes:(test set)

- movie_id: unique id for the movie.
- plot_summary: summary of the plot without any spoilers.
- duration: run time of the movie.
- genre: list of genres the movie belongs to
- rating: star rating out of 10.
- release_date: date the movie was released
- plot_synopsis: revealing details of the plot of the movie.

Each record in IMDB_reviews consists of the following attributes:(train and evaluation set)

- review_date: date the review was posted.
- movie_id: unique id of the movie for which the review is about.
- user_id: unique id of the user who left the review.
- is_spoiler: whether the review contains spoilers.
- review_text: text of the review.
- rating: star rating (out of 10) given by the user.
- review_summary: review summary provided by the user.

Source:IMDB Spoiler dataset column(kaggle data)

3 Loading dataset

```
import pandas as pd
import json

data = []
with open('IMDB_reviews.json', 'r') as file:
    for line in file:
        data.append(json.loads(line))

df = pd.DataFrame(data)

df
```

Figure 1: loading training set

4 EDA(Exploratory data analysis)

Dataset info:

```
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 573913 entries, 0 to 573912
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   review_date     573913 non-null object
1   movie_id        573913 non-null object
2   user_id         573913 non-null object
3   is_spoiler      573913 non-null bool
4   review_text     573913 non-null object
5   rating          573913 non-null object
6   review_summary  573913 non-null object
7   review_length   573913 non-null int64
dtypes: bool(1), int64(1), object(6)
memory usage: 31.2+ MB
```

Figure 2: dataset info.

Some description about training data:

- 1.Missing values
- 2.Spoiler VS non spoiler distribution
- 3.Review length description

Missing Values:	
Column	Missing Values
review_date	0
movie_id	0
user_id	0
is_spoiler	0
review_text	0
rating	0
review_summary	0

Spoiler vs. Non-Spoiler Distribution:	
Is Spoiler	Proportion
False	0.7370263437141169
True	0.26297365628588304

Review Length Description:	
Statistic	Value
count	573913.0
mean	1460.5535246631457
std	1125.577018615137
min	18.0
25%	719.0
50%	1052.0
75%	1815.0
max	14963.0

Figure 3: description

Conclusion:

- 1.No missing values found in any column.
- 2.Data needs downsampling of non-spoiler data since it dominates over minority class.

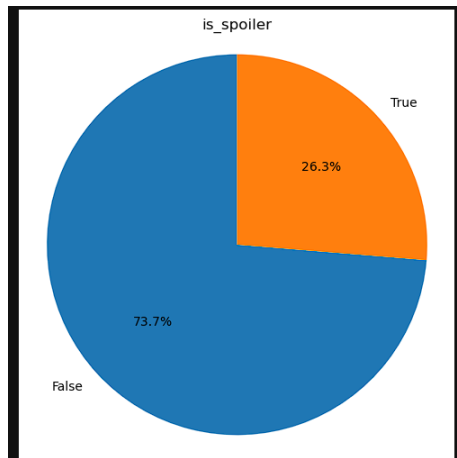


Figure 4: description

Genre distribution in spoiler reviews:

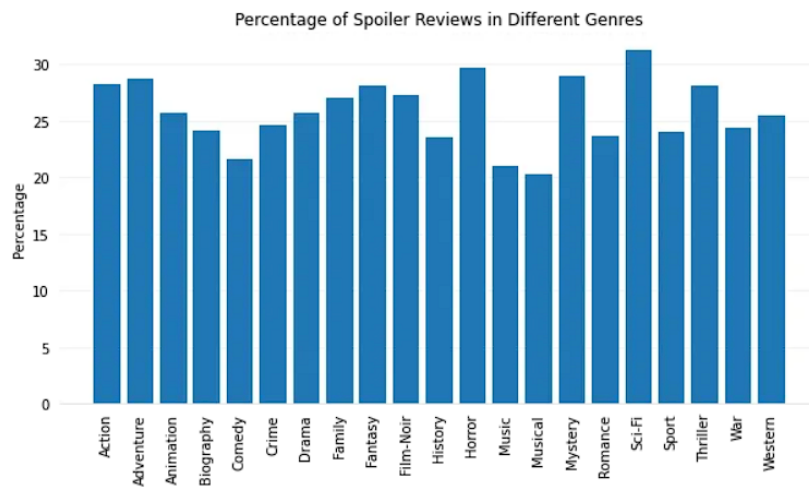


Figure 5: description

Spoiler trend over time:

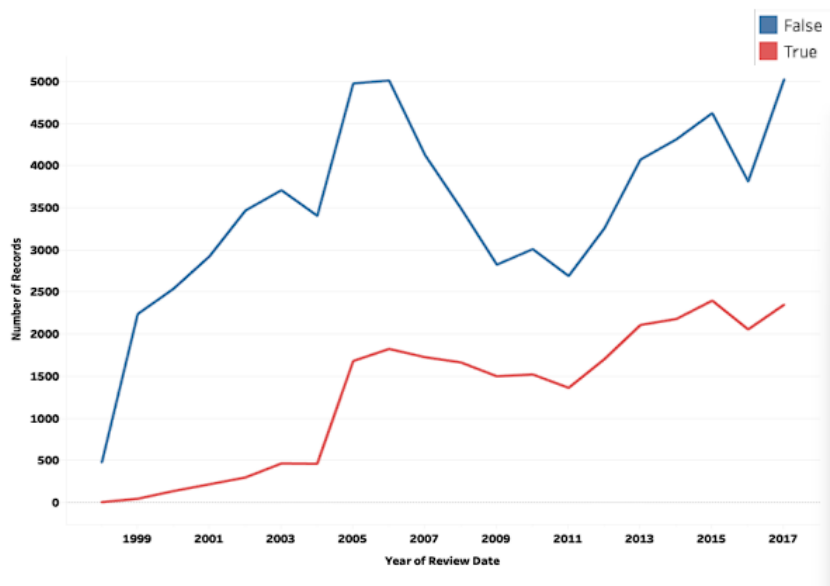


Figure 6: description

Review length vs frequency

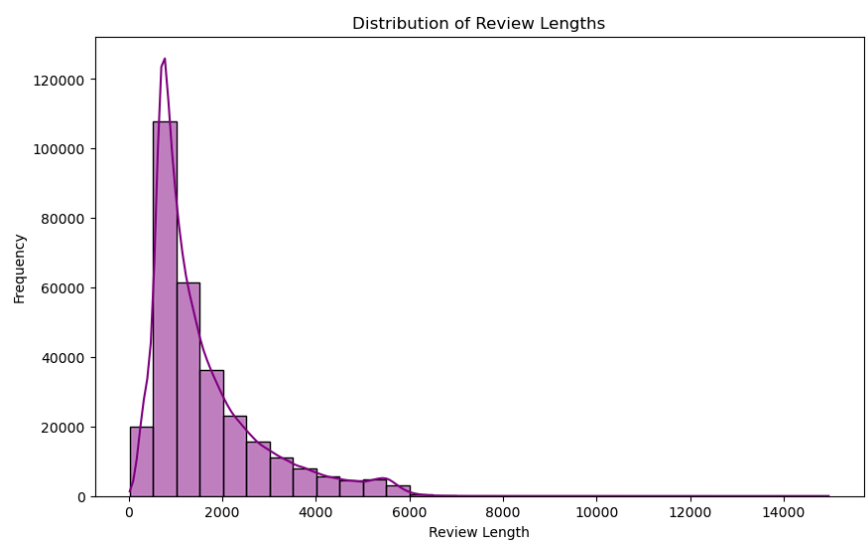


Figure 7: description

Box plot b/w review length and is_spoiler:

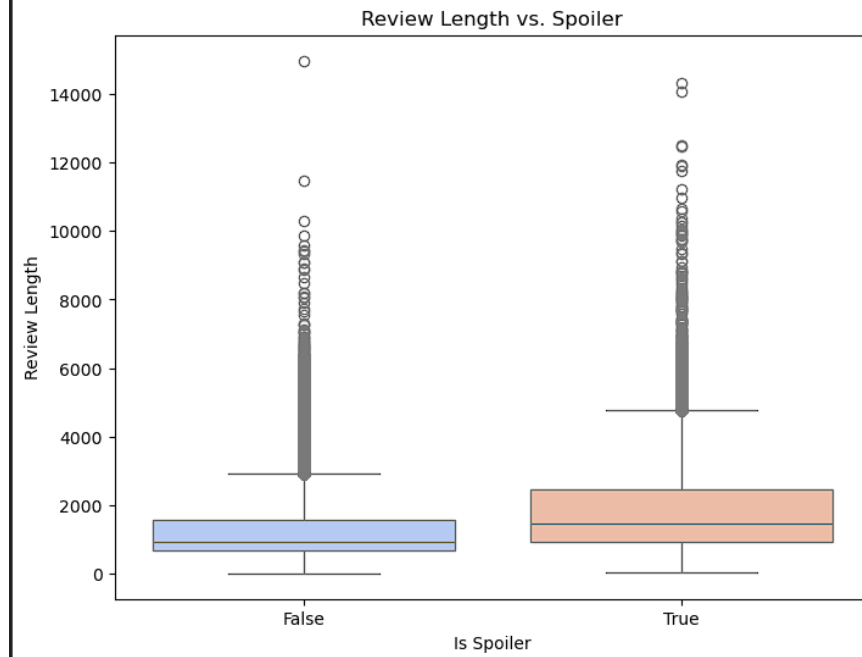


Figure 8: description

5 Methodology

Downsampling

In machine learning, downsampling refers to the process of reducing the number of samples in the majority class so that the class distribution becomes more balanced.

It's part of resampling strategies used to handle imbalanced datasets, which can otherwise bias the learning algorithm toward the more frequent class.

Problem of Class Imbalance

Imagine a binary classification task:

If you train a model on this dataset as-is, it might learn to always predict non-

Class	Number of samples
Non-Spoiler(0)	90,000
Spoiler(1)	10,000

Table 2: Imbalance positive-negative data

spoiler (0) to get high accuracy (90percent)—but it completely fails to detect

spoilers, which defeats the purpose.

This is called a class imbalance problem, and it can lead to:

- Poor recall for the minority class (false negatives).
- Misleading accuracy metrics.

What Does Downsampling Do? It fixes this by randomly removing samples from the majority class (non-spoilers in this case) so that both classes have similar sample sizes.

After downsampling:

Now the model sees an equal number of both classes during training, making

Class	Number of samples
Non-Spoiler(0)	10,000
Spoiler(1)	10,000

Table 3: balanced positive-negative data

it more likely to learn the decision boundary properly.

Pros	Cons
Simple and fast	Loses data from the majority class
Balances the dataset	May discard useful patterns
Reduces training time	Not suitable if the majority class is small
No synthetic data (unlike SMOTE)	May underfit if too much is removed

Table 4: Pros and cons of undersampling in imbalanced datasets

Preprocessing Summary

The following preprocessing steps are applied to the `review_text` column in order to create a cleaned version `cleaned_review_text` suitable for input into a BERT-based spoiler detection model:

1. Removing Noise:

- Elimination of URLs (e.g., `http://...`, `www...`)
- Removal of email addresses (e.g., `*@*`)
- Filtering of non-word characters and digits

2. Text Normalization:


```

is_spoiler
False    150924
True     150924
Name: count, dtype: int64

   review_date  movie_id  user_id  is_spoiler  \
0   5 August 2006  tt0415306  ur11723253    False
1   20 July 2007   tt0482571  ur8387182    False
2   3 January 2014  tt0113189  ur45796763    False
3   24 June 2012   tt0120201  ur1994077     True
4  13 December 2011  tt0170016  ur25851619     True

   review_text  rating  \
0  This was the funniest movie I've ever seen. Sa...    10
1  It's a long time since I saw a film where I wa...     9
2  Where to start? an absolute classic, a film th...    10
3  I haven't read Heinlein's original novel, so I...     5
4  After the 1966 Animated short people were in t...    10

   review_summary  review_length
0  "If you're not the winner, you're the loser," ...    886
1      Surprising and thought-provoking             749
2      Best Bond Film Ever                          964
3  And after Prom, we'll go teach those bad bugs ...   1670
4  Dr. Suess' How The Grinch Stole Christmas(2000).    873

```

Figure 9: Dataset after downsampling non-spoiler class.

- Conversion of all text to lowercase
- Normalization of whitespace

3. Tokenization:

- Splits text into individual tokens (words)

4. Stopword Removal and Stemming:

- Removes common English stopwords (e.g., “the”, “is”)
- Applies Porter Stemming to reduce words to their root forms (e.g., “running” → “run”)

5. Text Reconstruction:

- Joins the filtered and stemmed tokens back into a cleaned text string

6. Efficiency:

- Utilizes `tqdm` with `.progress_apply()` for monitoring the cleaning progress

Advantages

This preprocessing pipeline helps enhance both the **training efficiency** and **model accuracy** when used in transformer-based models like BERT.

```

#downsampling data for imbalance class is_spoiler
import pandas as pd
# Step 1: Identify the class counts
class_counts = df['is_spoiler'].value_counts()
min_class_count = class_counts.min() # Get the count of the minority class

# Step 2: Separate the classes
df_class_0 = df[df['is_spoiler'] == 0]
df_class_1 = df[df['is_spoiler'] == 1]

# Step 3: Downsample the majority class
df_class_0_downsampled = df_class_0.sample(min_class_count, random_state=42) # Use random_state for reproducibility
df_class_1_downsampled = df_class_1.sample(min_class_count, random_state=42) # Ensuring it's already the minority size

# Step 4: Concatenate the results
df_balanced = pd.concat([df_class_0_downsampled, df_class_1_downsampled])

# Step 5: Shuffle the dataset
df_balanced = df_balanced.sample(frac=1, random_state=42).reset_index(drop=True)

# Replace the initial DataFrame with the balanced DataFrame
df = df_balanced

# Display the new class distribution and a few rows to verify
print(df['is_spoiler'].value_counts())
print(df.head())

```

Figure 10: Code for downsampling.

Step	Benefit
Noise removal	Eliminates irrelevant content (URLs, emails, etc.)
Lowercasing & normalization	Standardizes text for uniform representation
Tokenization	Prepares text for NLP model input
Stopword removal	Reduces noise and dimensionality
Stemming	Groups word variants under a common root
TQDM progress bar	Improves usability during processing

```

def clean_text(text):
    # Remove URLs
    text = re.sub(r'http\S+', '', text)
    text = re.sub(r'www\S+', '', text)

    # Remove emails
    text = re.sub(r'\S*@\S*\s?', '', text)

    # Remove all non-word characters and digits
    text = re.sub(r'^a-zA-Z\s', '', text)

    # Normalize whitespaces
    text = re.sub(r'\s+', ' ', text)

    # Convert text to lowercase
    text = text.lower()

    # Tokenize text
    tokens = nltk.word_tokenize(text)

```

Figure 11: Preprocessing to make data uniform.

Applying SMOTE

SMOTE (Synthetic Minority Over-sampling Technique) is a popular method used in machine learning to handle imbalanced datasets, where one class (usually the one of interest) has significantly fewer samples than others.

In classification tasks, if one class dominates the dataset, most models get biased towards that majority class and perform poorly on the minority class.

How does it work:

For each sample in the minority class, SMOTE finds its k -nearest neighbors (usually $k = 5$) within the same class.

- It then randomly selects one of these neighbors.
- A synthetic sample is generated along the line connecting the original point and the selected neighbor.
- In this way, new data points are created through interpolation rather than by duplication.

Example:

Suppose we have a minority class sample A and its neighbor B :

$$A = (2, 3), \quad B = (4, 5)$$

A synthetic point might be computed as:

$$A_{\text{new}} = A + \delta \cdot (B - A), \quad \delta \in [0, 1]$$

If $\delta = 0.5$, then:

$$A_{\text{new}} = (2, 3) + 0.5 \cdot [(4, 5) - (2, 3)] = (3, 4)$$

Why smote was applied:

- Generates synthetic *spoiler* examples in the training set.
- Balances the label distribution, allowing the model to learn to detect spoiler patterns more effectively.
- Avoids overfitting that occurs with naive oversampling, since it does not simply duplicate the same samples.

Pros of SMOTE	Cons of SMOTE
Reduces overfitting compared to simple duplication.	Can introduce noise if the minority class is spread out or overlaps with other classes.
Helps the model learn decision boundaries better.	Not effective if the minority class has very few samples to begin with.

Table 5: Pros and Cons of SMOTE

```

from sklearn.feature_extraction.text import TfidfVectorizer
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tqdm import tqdm

# Encode the target label
label_encoder = LabelEncoder()
df['label'] = label_encoder.fit_transform(df['is_spoiler'])

# Split the data before applying SMOTE
train_df, temp_df = train_test_split(df, test_size=0.2, random_state=42, stratify=df['label'])

# Vectorization and SMOTE
vectorizer = TfidfVectorizer(max_features=10000) # reduce feature dimension drastically
smote = SMOTE(random_state=42)

# Create the pipeline with imblearn Pipeline
pipeline = Pipeline([
    ('vectorizer', vectorizer),
    ('smote', smote)
])

# Fit and transform with tqdm
for _ in tqdm(range(1), desc="Applying Vectorization and SMOTE"):
    X_resampled, y_resampled = pipeline.fit_resample(train_df['cleaned_review_text'], train_df['label'])

print("Resampled dataset shape:", X_resampled.shape)

# Convert back to DataFrame if needed
train_df_resampled = pd.DataFrame.sparse.from_spmatrix(X_resampled, columns=vectorizer.get_feature_names_out())
train_df_resampled['label'] = y_resampled

# Re-split the validation and test sets (no need to apply SMOTE here)
val_df, test_df = train_test_split(temp_df, test_size=0.5, random_state=42, stratify=temp_df['label'])

# Tokenization
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

```

Figure 12: Applying SMOTE

6 Modelling and Training the Data

BERT Model

BERT is a transformer-based pre-trained language model introduced by Google AI in their 2018 paper:

“BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”

It’s designed to understand the deep context of words in a sentence by looking both to the left and right simultaneously, unlike older models that were unidirectional.

6.1 Architecture overview

- **Transformer Encoder Only:**

- BERT uses only the encoder stack of the Transformer architecture (unlike GPT, which uses the decoder stack).
- Key components include:

- * **Self-attention:** Allows BERT to weigh the relevance of different words with respect to each other in a sentence.
- * **Multi-head attention:** Enables the model to capture multiple types of relationships between words simultaneously.
- * **Layer normalization, feedforward networks, and residual connections:** These components contribute to stable and efficient deep learning in BERT’s architecture.

Variant	Layers	Hidden Size	Attention Heads	Parameters
BERT-Base	12	768	12	110M
BERT-Large	24	1024	16	340M

Table 6: Comparison of BERT Variants

Pre-training Tasks

1. Masked Language Modeling (MLM)

- Randomly masks 15% of tokens in a sentence.
- The model learns to predict the masked words using context from both directions.
- **Example:**
 - **Input:** “The movie was [MASK].”
 - **Output:** “great”

2. Next Sentence Prediction (NSP)

- Trains BERT to understand relationships between sentences.
- **Input:** Pair of sentences A and B.
- 50% of the time B is the actual next sentence.
- 50% of the time it’s a random sentence.
- **Task:** Predict whether B is the next sentence of A.

Fine-tuning BERT

After pre-training, BERT is fine-tuned for specific tasks using labeled datasets by adding just a small task-specific layer on top.

- **Example: Spoiler Detection**
 - **Input:** Movie review text.

- **Model:** BERT + classification head.
- **Output:** Probability of the text being a spoiler.
- **Fine-tuning requires:**
 - Adding a [CLS] token at the beginning.
 - Using the final hidden state of the [CLS] token for classification.
 - A binary cross-entropy loss for binary tasks like spoiler detection.

Feature	Benefit
Bidirectional context	Understands nuanced meaning better.
Pre-trained on massive data	Generalizes well across domains.
Transfer learning	Works with minimal labeled data after fine-tuning.
State-of-the-art performance	Outperforms previous models in tasks like QA, NER, sentiment analysis.

Table 7: Key Benefits of BERT

Limitations

- **Compute-intensive:** Requires powerful GPUs or TPUs.
- **Slow inference:** Especially BERT-Large.
- **No causal generation:** Not designed for text generation like GPT.

Pipeline of model training

BERT-Based Spoiler Detection Pipeline

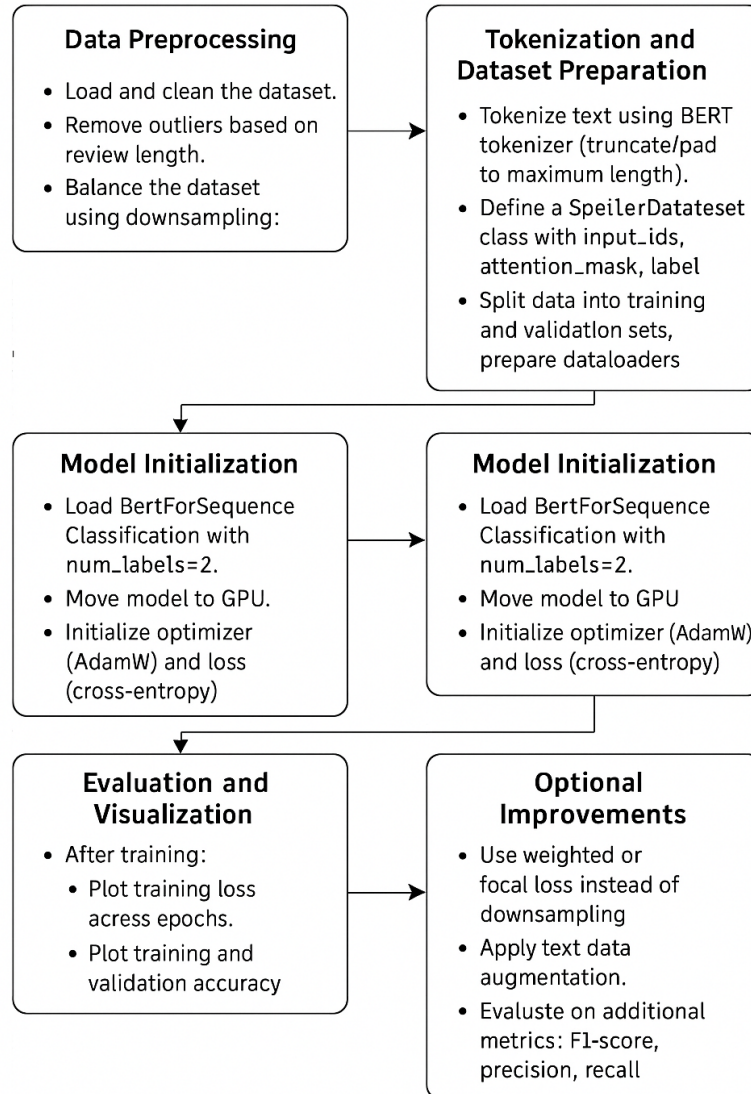


Figure 13: Pipeline

Training and Evaluation Functions

This section describes the implementation of the training and evaluation loops used for fine-tuning a BERT model to detect spoilers in movie reviews.

6.2 Training Function: `train()`

The `train()` function performs a full epoch of training on the provided dataloader.

Listing 1: Training Function

```
def train(model, dataloader, optimizer, loss_fn):
    model.train()
    total_loss, total_acc = 0, 0

    progress_bar = tqdm(dataloader, desc="Training")
    for batch in progress_bar:
        optimizer.zero_grad()
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['label'].long().to(device)

        outputs = model(input_ids, attention_mask=attention_mask, labels=labels)
        loss = outputs.loss
        logits = outputs.logits

        preds = torch.argmax(logits, dim=1)
        acc = (preds == labels).float().mean()

        loss.backward()
        optimizer.step()

        total_loss += loss.item()
        total_acc += acc

    progress_bar.set_postfix(
        loss=total_loss / len(dataloader),
        acc=total_acc / len(dataloader)
    )

    return total_loss / len(dataloader), total_acc / len(dataloader)
```

Explanation

- `model.train()` sets the model to training mode, enabling dropout and gradients.

- A progress bar is initialized to monitor training status.
- Each batch is moved to the computing device (CPU/GPU).
- The model performs a forward pass using the input tokens, attention masks, and labels.
- The cross-entropy loss and predicted logits are extracted.
- Predictions are compared to labels to compute batch accuracy.
- Backpropagation is performed using `loss.backward()` and `optimizer.step()`.
- Average loss and accuracy over the dataloader are returned.

6.3 Evaluation Function: `evaluate()`

The `evaluate()` function is used to assess model performance on the validation or test dataset without updating model weights.

Listing 2: Evaluation Function

```
def evaluate(model, dataloader):
    model.eval()
    total_acc = 0

    with torch.no_grad():
        progress_bar = tqdm(dataloader, desc="Evaluating")
        for batch in progress_bar:
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            labels = batch['label'].to(device)

            outputs = model(input_ids, attention_mask=attention_mask)
            logits = outputs.logits

            preds = torch.argmax(logits, dim=1)
            acc = accuracy_score(labels.cpu().numpy(), preds.cpu().numpy())

            total_acc += acc

        progress_bar.set_postfix(acc=total_acc / len(dataloader))

    return total_acc / len(dataloader)
```

Explanation

- `model.eval()` switches the model to evaluation mode, disabling dropout.

- `torch.no_grad()` context disables gradient calculation, saving memory and computation. Batches are removed from the dataloader.
- A forward pass is conducted to compute logits.
- Predictions are computed via `argmax`, and accuracy is measured using `accuracy_score()`.
- Average accuracy across all batches is returned.

6.4 Summary

Function	Purpose	Mode	Gradients	Returns
<code>train()</code>	Fine-tune model	Training	Enabled	Avg. loss & accuracy
<code>evaluate()</code>	Validate/Test model	Evaluation	Disabled	Avg. accuracy

Table 8: Comparison of training and evaluation loops

```
import matplotlib.pyplot as plt

def train(model, dataloader, optimizer, loss_fn):
    model.train()
    total_loss, total_acc = 0, 0

    progress_bar = tqdm(dataloader, desc="Training")
    for batch in progress_bar:
        optimizer.zero_grad()
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['label'].long().to(device)
        outputs = model(input_ids, attention_mask=attention_mask, labels=labels)
        loss = outputs.loss # Uses CrossEntropyLoss internally
        logits = outputs.logits
        preds = torch.argmax(logits, dim=-1)
        acc = (preds == labels).float().mean()

        loss.backward()
        optimizer.step()

        total_loss += loss.item()
        total_acc += acc

    progress_bar.set_postfix(loss=total_loss / (len(dataloader)), acc=total_acc / (len(dataloader)))

    return total_loss / len(dataloader), total_acc / len(dataloader)

def evaluate(model, dataloader):
    model.eval()
    total_acc = 0

    with torch.no_grad():
        progress_bar = tqdm(dataloader, desc="Evaluating")
        for batch in progress_bar:
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            labels = batch['label'].to(device)
            outputs = model(input_ids, attention_mask=attention_mask)
            preds = torch.argmax(outputs.logits, dim=-1)
            acc = accuracy_score(labels.cpu().numpy(), preds.cpu().numpy())

            total_acc += acc

        progress_bar.set_postfix(acc=total_acc / (len(dataloader)))

    return total_acc / len(dataloader)
```

Figure 14: train and test functions.

Custom Dataset Class: SpoilerDataset

The `SpoilerDataset` class is a custom PyTorch dataset designed to handle the spoiler detection data. It inherits from the `torch.utils.data.Dataset` class and enables the `DataLoader` to iterate over input samples and corresponding labels efficiently.

Listing 3: SpoilerDataset Class Definition

```
class SpoilerDataset(Dataset):
    def __init__(self, texts, labels):
        self.texts = texts
        self.labels = labels

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        text = self.texts[idx]
        label = self.labels[idx]

        encoding = tokenizer(
            text,
            truncation=True,
            padding='max_length',
            max_length=512,
            return_tensors='pt'
        )

        return {
            'input_ids': encoding['input_ids'].squeeze(0),
            'attention_mask': encoding['attention_mask'].squeeze(0),
            'label': torch.tensor(label, dtype=torch.long)
        }
```

Description of Methods

- **__init__**: Initializes the dataset object with two arguments:
 - **texts**: A list of input text samples (e.g., movie reviews).
 - **labels**: A list of corresponding binary labels (e.g., 0 for non-spoiler, 1 for spoiler).
- **__len__**: Returns the total number of samples in the dataset, allowing PyTorch to iterate through the dataset.
- **__getitem__**:

- Retrieves the text and label at the given index.
- Applies the BERT tokenizer to the text with padding and truncation to a maximum length of 512 tokens.
- Converts the tokenized outputs into tensor format required for BERT.
- Returns a dictionary containing:
 - * **input_ids**: Tensor of token IDs.
 - * **attention_mask**: Tensor that indicates which tokens are padding (0) and which are real input (1).
 - * **label**: Ground truth tensor.

Advantages

- The class provides a standardized way to tokenize and organize inputs for the BERT model.
- Ensures consistent input shape using truncation and padding.
- Integrates seamlessly with `DataLoader` for batching and shuffling.

```
import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
bert_model = bert_model.to(device)
import torch.nn as nn

loss_fn = nn.CrossEntropyLoss()

# Lists to store metrics
train_losses = []
train_accuracies = []
val_accuracies = []

n_epochs = 5 # Set the number of epochs
for epoch in range(n_epochs):
    print(f"Epoch {epoch+1}/{n_epochs}")

    train_loss, train_acc = train(bert_model, train_loader, optimizer, loss_fn)
    val_acc = evaluate(bert_model, val_loader)

    # Append metrics
    train_losses.append(train_loss)
    train_accuracies.append(train_acc)
    val_accuracies.append(val_acc)

    print(f"Epoch {epoch+1}: Train Loss {train_loss:.4f}, Train Acc {train_acc:.4f}, Val Acc {val_acc:.4f}")
```

Figure 15: Bert Model training-testing

7 Results and accuracy

BERT Model Metric	Value
Accuracy	0.6777
F1 Score	0.6327

Link to code:Attached code